SHA

```python
import hashlib

message = input("Enter message: ").encode()

hashedValue = hashlib.sha256(message).hexdigest()

print("Hashed message:", hashedValue)
```

HMAC

```python
import hmac
import hashlib

key = input("Enter key: ").encode()
message = input("Enter message: ").encode()

hmacValue = hmac.new(key, message, hashlib.sha256).hexdigest()

print("Hmac Value:", hmacValue)
```

Diffie Hellman

```python
# Remember these formulas:

# Public values: prime p, base g

# Alice picks private a -> A = (g ** a) % p
# Bob picks private b   -> B = (g ** b) % p

# Shared key:
#     Alice: K = (B ** a) % p
#     Bob:   K = (A ** b) % p

p = int(input("Enter prime p: "))
g = int(input("Enter base g: "))

a = int(input("Alice private key (a): "))
A = (g ** a) % p

b = int(input("Bob private key (b): "))
B = (g ** b) % p

print("\nAlice sends:", A)
print("Bob sends:", B)

keyAlice = (B ** a) % p
keyBob = (A ** b) % p

print("\nShared key (Alice):", keyAlice)
print("Shared key (Bob):", keyBob)
```

```python
# Remember these formulas (RSA)

# n = p * q
# phi (n) = (p-1) * (q-1)

# Choose e such that gcd (e, phi(n)) = 1
# Choose d such that (d * e) % phi = 1 (remainder is 1)

# Encryption: c = (m ** e) % n
# Decryption: m = (c ** d) % n

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d

p = int(input("Enter prime p: "))
q = int(input("Enter prime q: "))

n = p * q
phi = (p - 1) * (q - 1)

e = 2
while gcd(e, phi) != 1:
    e += 1

d = mod_inverse(e, phi)

print("\nPublic Key  (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

m = int(input("\nEnter message (number): "))

c = (m ** e) % n
print("Encrypted:", c)

decrypted = (c ** d) % n
print("Decrypted:", decrypted)
```

AES

```python
# Run this in the terminal first: pip install cryptography

from cryptography.fernet import Fernet
import hashlib
import base64

k = input("Enter key: ").encode()

key = base64.urlsafe_b64encode(hashlib.sha256(k).digest())

f = Fernet(key)

message = input("Plaintext: ").encode()

ciphertext = f.encrypt(message).decode()
plaintext = f.decrypt(ciphertext).decode()

print("Encrypted:", ciphertext)
print("Decrypted:", plaintext)
```

Caesar Cipher

```python
key = int(input("Key: ")) # Only Numbers!
text = input("Text: ")

function = lambda s, k: ''.join (
    chr ((ord(c) - 65 + k) % 26 + 65) if c.isupper() else
    chr ((ord(c) - 97 + k) % 26 + 97) if c.islower() else c
    for c in s
)

ciphertext = function (text, key)
decrypted = function (ciphertext, -key)

print("Ciphertext:", ciphertext)
print("Decrypted:", decrypted)
```

Vernam Cipher

```python
plaintext = input("Enter plaintext: ")
key = input("Enter key (same length as plaintext): ")

ciphertext = ""
for p, k in zip(plaintext, key):
    cipher += chr(ord(p) ^ ord(k))

print("Encrypted:", cipher.encode().hex())

decrypted = ""
for c, k in zip(ciphertext, key):
    decrypted += chr(ord(c) ^ ord(k))

print("Decrypted:", decrypted)
```

Railfence Cipher

```python
key = int(input("Enter key: ")) # Only Numbers

text = input("Enter plaintext: ")

# Encrypt
rows = [''] * key
i, step = 0, 1

for c in text:
    rows[i] += c
    if i == 0: step = 1
    elif i == key - 1: step = -1
    i += step

c = ''.join(rows)
print("Encrypted:", c)

# Decrypt
res = [''] * len(text)
pattern = []

i, step = 0, 1
for _ in text:
    pattern.append(i)
    if i == 0: step = 1
    elif i == key - 1: step = -1
    i += step

pos = 0
rows = [''] * key
for r in range(key):
    count = pattern.count(r)
    rows[r] = c[pos:pos+count]
    pos += count

idx = [0] * key
for i in range(len(text)):
    r = pattern[i]
    res[i] = rows[r][idx[r]]
    idx[r] += 1
```

```
d = ''.join(res)
print("Decrypted:", d)
```

```python
# Columnar Cipher

key = input("Enter key: ")
text = input("Enter text: ")

n = len(key)

rows = [text[i:i+n] for i in range(0, len(text), n)]

order = sorted(range(n), key=lambda i: key[i])

c = ''
for i in order:
    for r in rows:
        if i < len(r):
            c += r[i]

print("Encrypted:", c)


col_len = len(text) // n
extra = len(text) % n

cols = {}
pos = 0

for idx, i in enumerate(order):
    l = col_len + (1 if idx < extra else 0)
    cols[i] = c[pos:pos+l]
    pos += l

num_rows = (len(text) + n - 1) // n

res = ''
for i in range(num_rows):
    for j in range(n):
        if i < len(cols[j]):
            res += cols[j][i]

print("Decrypted:", res)
```

Playfair cipher

```
def generate_matrix (key):

    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"  # no J here
    key = key.upper().replace('J', 'I')

    result = ""
    for ch in key + alphabet:
        if ch not in result and ch.isalpha():
            result += ch

    return [result[i:i+5] for i in range(0, 25, 5)]

def find_position(matrix, char):
    for row in range(5):
        for col in range(5):
            if matrix[row][col] == char:
                return row, col

def make_pairs(text):
    text = text.upper().replace('J', 'I')
    pairs = []
    i = 0

    while i < len(text):
        a = text[i]
        b = text[i+1] if i+1 < len(text) else 'X'

        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2

    return pairs

def encrypt (plaintext, key):
    matrix = generate_matrix(key)
    pairs = make_pairs(plaintext)
    ciphertext = ""
```

```
    for a, b in pairs:
        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)

        if r1 == r2:  # same row -> go right
            ciphertext += matrix[r1][(c1 + 1) % 5]
            ciphertext += matrix[r2][(c2 + 1) % 5]

        elif c1 == c2:  # same column -> go down
            ciphertext += matrix[(r1 + 1) % 5][c1]
            ciphertext += matrix[(r2 + 1) % 5][c2]

        else:  # rectangle -> swap columns
            ciphertext += matrix[r1][c2]
            ciphertext += matrix[r2][c1]

    return ciphertext

def decrypt (ciphertext, key):
    matrix = generate_matrix(key)
    plaintext = ""

    for i in range(0, len(ciphertext), 2):
        a, b = ciphertext[i], ciphertext[i+1]

        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)

        if r1 == r2:  # same row -> go left
            plaintext += matrix[r1][(c1 - 1) % 5]
            plaintext += matrix[r2][(c2 - 1) % 5]

        elif c1 == c2:  # same column -> go up
            plaintext += matrix[(r1 - 1) % 5][c1]
            plaintext += matrix[(r2 - 1) % 5][c2]

        else:  # rectangle -> swap columns
            plaintext += matrix[r1][c2]
            plaintext += matrix[r2][c1]

    return plaintext
```

```
key = input("Enter key: ")
plaintext = input("Enter plaintext: ")

ciphertext = encrypt (plaintext, key)

print("Encrypted:", ciphertext)
print("Decrypted:", decrypt (ciphertext, key))
```